

AAI/CPE/EE 551 Course Project

Overall Requirements

- Identify a real-world engineering or science problem and write a Python program to solve the problem with correct results.
- Use Jupyter Notebook for the main program, and name it `main.ipynb`. Use VS Code for all other functions and classes, which should be divided into multiple `.py` files, each serving as a module. The `main.ipynb` imports the modules as needed.
- Use publicly available datasets with no IP issues. The final project submission must include the data source information. Alternatively, you can create your own dataset.
- Use Python version 3.14, 3.13, or 3.12.
- It's a team project; 2-3 members per team. Team members need to make equal contributions.
- Use GitHub as the code repository and add all group members to it. Each group member should make at least five (5) meaningful commits to the group repository during the development. The meaningful commits contribute to program logic, design, debugging, testing, data handling, or documentation. Cosmetic edits (formatting/spacing/renaming only) do not count.
- The README.md file must have all the detailed information about the project, including:
 - Project title
 - Project description, including project overview and functionalities
 - Project file/module structure
 - Project dependencies/environments/libraries, and how to install them on Windows or Mac. For Jupyter Notebook, specify the kernel information
 - How the project requirements (both parts 1 and 2) and grading rubrics (on page 3) are met
 - **Step-by-step and detailed instructions on how to run the program on Windows or Mac.**
 - Team member names, emails, and Stevens IDs
 - Main contributions of each team member

Requirements (Part 1): your program must have all the following 10 components

1. Have at least two meaningful, well-defined classes that have constructors, attributes, methods, and instance objects. These two classes must have a relationship: either inheritance or composition. Meaningful classes contain functional logic relevant to the problem (not empty or trivial structures).
2. Define at least two meaningful, well-defined functions that contribute directly to solving the identified problem.
3. Use at least two advanced Python libraries for data and processing, such as matplotlib, PyTorch, NumPy, or Pandas. The libraries must provide critical functionality to the program and cannot be used in a superfluous way.
4. Include at least two distinct exception-handling scenarios (e.g., invalid input, file/data loading errors, type errors, runtime/math errors) and at least two functional Pytest test cases validating program logic.
5. Perform some meaningful data I/O, such as reading from a file or from a database, etc.
6. Use at least two loop statements (for or while) and at least two if statements demonstrating logical control flow.
7. Use at least two mutable types (list, dict, set) and two immutable types (number, string, tuple).
8. Implement `__str__()` and at least one additional operator overload, e.g., `__add__()`, `__len__()`, or `__eq__()`.

9. Include a proper docstring, header information, and meaningful comments for each module, class, and function. Docstrings must be placed immediately below the definition (class, function) or at the very beginning of a module.
10. `__name__`

Requirements (Part 2): your program must have at least four components from the following:

1. Comprehension of at least one core data type, e.g., list comprehension
2. One built-in library/module
3. One generator function or generator expression
4. One special function, e.g., map, zip, filter, lambda, or reduce
5. Set operations
6. Recursion
7. Closure
8. Decorator
9. `getattr()` or `__getattr__()`
10. One linear data structure (array, linked List, stack, or queue)

The README.md file needs to list which four components are included in your program.

Important Notices

- Do not use any AI tools to design the project.
- Do not copy any existing projects; Do not download code from the Internet.
- Any discovered instances of these will be considered cheating, and appropriate actions will be taken according to the course syllabus.
- Be sure you have tested the program you wish to submit and that it works correctly. You will not be allowed to resubmit work after the deadline.

Submissions

- **July 8: project proposal submission (10 points)** -- one submission per team (only one member submits the proposal to Canvas).

Submit a 2-page .pdf file describing the engineering/science problem that you propose to solve and the plan on how you will meet the requirements (both part 1 and part 2).

Proposal format:

- Project title
- Team member names, emails, and Stevens IDs
- Section 1: Project description
- Section 2: Plan to meet the Requirements Part 1
- Section 3: Plan to meet the Requirements Part 2
- Section 4: Timeline

- **August 24: project final submission (90 points)** -- one submission per team (only one member submits the proposal to Canvas).

Submit the GitHub repository link to Canvas. Only submit the GitHub repository link.

Notice that: 1) You must set the GitHub repository to public, and it must include all the files needed to run your program and solve the identified problem, and 2) You must provide step-by-

step and detailed instructions in the README.md file – refer to the Overall Requirements. As an example, **below is a bad instruction** on how to run the program:

1. Download the repository
2. Install the required libraries
3. Open the main notebook
4. Run the notebook cells

Please keep in mind that the grader will exactly follow the instructions on how to run the program in your README.md file when grading your project submissions.

- Final project submissions will not be accepted if they do not meet the **Overall Requirements**.
- Late submissions are not accepted.

Grading Rubrics

- Your final project submission will be graded largely on whether it works correctly and meets all the requirements. Moreover, it will be graded based on your program style. This means that you should use docstrings, header information, comments, and meaningful variable names that follow the name conventions.
- You must submit all the files needed to execute your project. If a required file is not included and the grader cannot execute and test your program, you will lose significant points.
- Final project submission (90 points)

Correctness and requirements (55 points)

- Program runs correctly without errors (20 points). The project should execute successfully. A working program is the foundation for all other grading.
- Part 1 requirements (25 points). Includes classes, functions, exception handling, operator overloading, data I/O, loops, use of libraries, and docstrings. Each of these elements demonstrates fundamental Python programming skills.
- Part 2 requirements (10 points). The project must integrate at least four advanced features, such as comprehensions, generators, recursions, closures, or built-in modules.

Code quality and style (20 points)

- Proper code structure (5 points). Projects should separate code into meaningful files and modules to demonstrate good organization.
- Meaningful variable names and consistent formatting (5 points). Variables and formatting should be clear and professional, making the code easy to follow. All names should follow the name conventions.
- Clear docstrings, header information, and meaningful comments (5 points). Every function and class should have explanations so others can understand the purpose/functionalities.
- Logical program flow (5 points). Code should be easy to read and logically structured, showing thoughtful design.

Documentation and presentation (15 points)

- A comprehensive and easy-to-follow README.md file (5 points). Users should be able to run the project easily by following the README.md file.
- Explanation of problem and solution approach (5 points). The submission should explain what problem is being solved and how the program addresses it.
- Proper file submission (5 points). All files must be complete, well-named, and organized so the graders can run and review the project without difficulty.
- Deductions:
 - –30 points if it is not a team project.

- –20 points if the project cannot be executed due to missing files or errors in submission.
- –10 points if the README.md file is missing or lacks detailed step-by-step information.
- –10 points for failing to meet any major requirement, e.g., no classes, no exception handling, no use of advanced libraries.
- –10 points if it is determined that a team member did not make the required contributions.
- –10 points for poor programming style (name conventions, unclear variable names, lack of comments, no docstrings or header information).

Sample projects:

- Movie Recommendation and Rating Analysis System – A program that recommends movies based on user ratings and analyzes viewing patterns or genre popularity.
- Banking Transaction Management and Fraud Detection (Basic Simulation) – A system that manages deposits, withdrawals, and transfers while identifying unusual or suspicious activity.
- Traffic Flow Simulation and Analysis – A simulation that models intersection traffic using loops and conditional logic, analyzing efficiency under different timing conditions.
- HR employee management system – titles, positions, salaries, etc.
- Library book checking in/out, analysis, and prediction.
- Superstore (Costco, Walmart, ...) inventory management system, analysis, and prediction.
- Computer network routing, prediction, and analysis.
- Chess game analyzer.
- Stock portfolio tracker and prediction.
- Web scraping and analyzing the data of a website.
- Roomba/Robotics/Drone movement and mission accomplishment.

Notice that these are examples only; students are encouraged to propose other appropriate engineering/science problems.

Project Submission Checklist

- Jupyter Notebook runs the main program without errors
- All required Python modules/files are uploaded to your GitHub repository
- All requirements are satisfied
- Exceptions handled
- Tests passing
- README.md complete

Teammate Formation

To facilitate the team formations, we created a Google Spreadsheet. Here is the Spreadsheet link: https://docs.google.com/spreadsheets/d/1dsEAYD_iyjm5nqLH7qNeKNfr8xChvASe/edit?gid=583669119#gid=583669119

Please first enter your information in Section B as soon as possible. Then proactively reach out to classmates in Section B to form teams. Once a team is formed, you move their names to Section A. Eventually, Section B should be empty 5 days before the proposal deadline.

Please know that one of the purposes of making it a team project is to help you learn how to collaborate on a coding project. So please do your best to find a teammate. Otherwise, the instructor will randomly assign you a teammate 5 days before the proposal deadline.